



SÍLABO OFICIAL

I. INFORMACIÓN GENERAL

Carrera: Ingeniería de Software

Curso: Estructuras de Datos

Código: SI400

Ciclo: Cuarto

Créditos: 4

Horas semanales: 4 horas (teoría y práctica)

Prerrequisito: Programación Orientada a Objetos

II. MISIÓN Y VISIÓN

Misión: Formar líderes íntegros e innovadores con visión global para que transformen el Perú.

Visión: Ser líder en la educación superior por su excelencia académica y su capacidad de innovación.

III. LOGRO DEL CURSO

Al finalizar el curso, el estudiante implementa soluciones de software eficientes mediante el uso de estructuras de datos lineales, no lineales y algoritmos de búsqueda, optimizando el manejo de la memoria y el tiempo de ejecución para resolver problemas complejos de ingeniería de software.

Competencia General: Pensamiento Crítico (Nivel 2).

Competencia Específica: (ABET 1) Capacidad de identificar, formular y resolver problemas complejos de ingeniería aplicando principios de ingeniería, ciencia y matemáticas (Nivel 2).

IV. UNIDADES DE APRENDIZAJE

UNIDAD 1: ESTRUCTURAS DE DATOS LINEALES Y MEMORIA DINÁMICA

Logro: El estudiante implementa estructuras de datos lineales como pilas y colas utilizando punteros y gestión de memoria dinámica.

Temario:

1.1 Introducción a la complejidad algorítmica (Notación Big O).

1.2 Gestión de memoria dinámica y punteros.

1.3 Pilas (Stacks): Operaciones y aplicaciones.

1.4 Colas (Queues): Simples, circulares y de prioridad.

UNIDAD 2: ESTRUCTURAS NO LINEALES: ÁRBOLES

Logro: El estudiante construye jerarquías de datos utilizando árboles para organizar información de manera eficiente.

Temario:

2.1 Definición y terminología de árboles.

2.2 Árboles Binarios de Búsqueda (BST).

2.3 Recorridos: Pre-orden, In-orden y Post-orden.

2.4 Árboles Balanceados (AVL).



UNIDAD 3: ESTRUCTURAS NO LINEALES: GRAFOS

Logro: El estudiante modela problemas de interconexión compleja mediante el uso de grafos y algoritmos de recorrido.

Temario:

3.1 Definición de grafos: Dirigidos y no dirigidos.

3.2 Representación: Matriz de adyacencia y lista de adyacencia.

3.3 Algoritmos de recorrido: BFS (Búsqueda en amplitud) y DFS (Búsqueda en profundidad).

3.4 Caminos mínimos y algoritmos fundamentales (Dijkstra).

UNIDAD 4: ALGORITMOS DE BÚSQUEDA Y ORDENAMIENTO AVANZADO

Logro: El estudiante aplica técnicas de búsqueda y ordenamiento optimizadas para la recuperación de datos.

Temario:

4.1 Algoritmos de búsqueda: Lineal, binaria y por hashing.

4.2 Tablas Hash: Funciones de dispersión y resolución de colisiones.

4.3 Algoritmos de ordenamiento complejo: Quicksort, Mergesort y Heapsort.

V. METODOLOGÍA

El curso se desarrolla bajo el modelo educativo de la UPC, centrado en el estudiante. Se utilizan metodologías activas como el Aprendizaje Basado en Problemas (ABP). Las sesiones incluyen teoría para fundamentar conceptos y laboratorios prácticos donde se codifican soluciones en lenguajes de alto nivel (C++/Java). Se fomenta el trabajo colaborativo y la investigación autónoma para la resolución de retos de programación.

VI. EVALUACIÓN

Fórmula de calificación:

$$\text{Final} = (\text{PC1} \times 0.15) + (\text{EA1} \times 0.25) + (\text{PC2} \times 0.15) + (\text{DD1} \times 0.20) + (\text{EB1} \times 0.25)$$

Desglose:

PC1: Práctica Calificada 1 (Unidad 1)

EA1: Examen Parcial (Unidades 1 y 2)

PC2: Práctica Calificada 2 (Unidad 3)

DD1: Evaluación de Desempeño (Promedio de laboratorios, retos y participación)

EB1: Examen Final (Todas las unidades)

VII. BIBLIOGRAFÍA

1. Weiss, M. A. (2014). Data structures and algorithm analysis in C++. Pearson Education.
2. Sedgewick, R., & Wayne, K. (2011). Algorithms. Addison-Wesley Professional.
3. Cairo, O., & Guardati, S. (2006). Estructuras de datos. McGraw-Hill.
4. Cormen, T. H. (2009). Introduction to algorithms. MIT Press.